PRACTICAL LEARNING GUIDE

Generative AI & Large Language Models

From transformers to production agents — theory you can explain, and code you can run.

Transformers | Tokens & Embeddings | Prompt Engineering | RAG | Vector Databases | LangChain | AI Agents | Responsible AI

Prepared for Sahil · Self-study companion · Theory + Practical · 8 modules

What’s inside

How to use this guide. Each module has two halves — **Theory** to build the mental model, then **Practical** code you can type out and run. Work one module per sitting. Read the theory first, then run every snippet and change one thing to see what breaks. The coloured boxes flag key ideas, things to try, and common pitfalls.

MODULE 01

Transformers Overview

> *Almost every model you'll hear about — GPT, Claude, Gemini, LLaMA, BERT — is a transformer. Understand this one architecture and the rest of the field stops feeling like magic.*

IN THIS MODULE

The problem transformers solved · Attention in plain English · The architecture (encoder / decoder) · Why they scale · **Practical:** a tiny self-attention from scratch in NumPy

Theory

The problem they solved

Before 2017, sequence models (RNNs, LSTMs) read text one word at a time, left to right, carrying a "memory" forward. Two things hurt: they were **slow** (each step waits for the previous one, so you can't parallelise across the sentence), and they **forgot** long-range context (information from word 1 got diluted by the time you reached word 50).

The 2017 paper *"Attention Is All You Need"* threw out recurrence entirely. A transformer looks at **all tokens at once** and lets each token directly "attend to" every other token. This made training massively parallel (great for GPUs) and gave every word direct access to every other word, no matter how far apart.

> Analogy
> An RNN is like reading a book through a keyhole, one word at a time, trying to remember everything you've seen. A transformer is like laying the whole page flat on a table — every word can glance at every other word instantly and decide which ones matter.

Attention, in plain English

Attention answers one question for each word: *"To understand me, which other words should I look at, and how much?"*

Take the sentence *"The animal didn't cross the street because it was tired."* What does **it** refer to? A human instantly links "it" to "animal." Attention lets the model do the same: when processing "it," it puts high weight on "animal" and low weight on "street."

Mechanically, every token produces three vectors:

- **Query (Q)** — what this token is looking for. ("I'm a pronoun, I need a noun.")
- **Key (K)** — what this token offers as a match. ("I'm a noun, an animal.")
- **Value (V)** — the actual information this token will pass along if attended to.

For each token, you compare its **Query** against every token's **Key** (a dot product = similarity score), turn those scores into weights with softmax, then take a weighted sum of the **Values**. The famous formula is just that:

> **THE ONE EQUATION TO REMEMBER**
>
> **Attention(Q, K, V) = softmax($Q \cdot K^T / \sqrt{d_k}$) · V**
>
> $Q \cdot K^T$ = how well each query matches each key. Dividing by $\sqrt{d_k}$ keeps the numbers stable. Softmax turns them into weights that sum to 1. Multiply by V to blend the information.

Multi-head attention

One attention calculation captures one kind of relationship. Real language has many at once: grammar, subject–object links, tone. So transformers run several attention calculations in parallel — called **heads** — each learning to focus on a different pattern, then concatenate the results. Think of it as several experts reading the same sentence, each tracking a different thread, then pooling notes.

The full architecture

A transformer block stacks a few pieces, and you repeat the block many times (GPT-3 has 96 layers). Each block contains:

Component	What it does
Input embedding	Turns each token into a vector (Module 2 covers this).
Positional encoding	Attention has no built-in sense of order, so we add a signal that encodes each token's position. Otherwise "dog bites man" = "man bites dog."
Multi-head self-attention	Every token gathers context from every other token.
Feed-forward network	A small MLP applied to each position — where much of the "knowledge" lives.
Residual + LayerNorm	Add the input back and normalise, so deep stacks train stably.

Three flavours you should recognise

Type	Example	Good at
Encoder-only	BERT	Understanding text: classification, search, embeddings. Reads whole input at once.
Decoder-only	GPT, Claude, LLaMA	Generating text. Predicts the next token, masked so it can't peek ahead.
Encoder–decoder	T5, original Transformer	Transforming one sequence into another: translation, summarisation.

The LLMs you'll mostly work with are **decoder-only**. They do one deceptively simple thing over and over: given the tokens so far, predict the next one. Everything impressive emerges from doing that extremely well at scale.

> **WHY THEY SCALE**
>
> Because there's no recurrence, the whole sequence trains in parallel. Add more layers, more heads, more data, more compute — and performance keeps improving predictably. Those *scaling laws* are the reason the industry keeps building bigger models.

Practical

Let's make attention concrete. We'll implement **scaled dot-product self-attention** from scratch with NumPy for a 3-word "sentence." No frameworks — just the equation above, so you can watch the weights form.

PRACTICAL 1.1 — SELF-ATTENTION FROM SCRATCH

```python
import numpy as np

# Pretend we already embedded 3 tokens as 4-dim vectors.
# Rows = tokens ["The", "cat", "sat"], columns = features.
X = np.array([
    [1.0, 0.0, 1.0, 0.0],   # The
    [0.0, 2.0, 0.0, 2.0],   # cat
    [1.0, 1.0, 1.0, 1.0],   # sat
])

# Learnable weight matrices (random here; a real model trains these).
np.random.seed(42)
W_q = np.random.randn(4, 4)
W_k = np.random.randn(4, 4)
W_v = np.random.randn(4, 4)

# Project each token into Query, Key, Value spaces.
Q = X @ W_q
K = X @ W_k
V = X @ W_v

def softmax(x):
    e = np.exp(x - x.max(axis=-1, keepdims=True))  # stable
    return e / e.sum(axis=-1, keepdims=True)

# 1) Scores: how much each token attends to every other token.
d_k = Q.shape[-1]
scores = (Q @ K.T) / np.sqrt(d_k)

# 2) Weights: softmax over each row so they sum to 1.
weights = softmax(scores)

# 3) Output: weighted blend of the Values.
output = weights @ V

print("Attention weights (rows sum to 1):")
print(np.round(weights, 2))
print("\nEach row of output is a context-aware vector:")
print(np.round(output, 2))
```

Run it. The `weights` matrix is the whole point: row *i* tells you how much token *i* pulled from each other token. Every row sums to 1 — that's the softmax. The `output` rows are the same three tokens, but each is now a blend that carries context from the others.

Try this

1. Print `scores` before softmax and compare to `weights` — watch softmax squash them into a probability distribution.
2. Change a row of `X` to be identical to another and see the weights shift.
3. Add a 4th token and confirm the weight matrix grows to 4×4. This quadratic growth (n^2) is exactly why long context is expensive.

Common misconception

"Attention weights are the model's explanation." Not quite. High attention to a word doesn't guarantee that word *caused* the output — attention is one signal among many across dozens of layers and heads. Useful for intuition; unreliable as proof.

Module 1 recap

- Transformers replaced step-by-step recurrence with **attention**, making training parallel and long-range context easy.
- Attention = for each token, compare its **Query** to all **Keys**, softmax the scores, blend the **Values**.
- **Multi-head** attention runs many such comparisons in parallel; stacking blocks with feed-forward layers, residuals and positional encoding gives the full model.
- Most LLMs are **decoder-only** next-token predictors. They scale predictably, which is why models keep getting bigger and better.

MODULE 02

Tokens & Embeddings

> *Models don't read words — they read numbers. This module is the bridge: how text becomes tokens, and how tokens become vectors that carry meaning.*

IN THIS MODULE

What a token is · Why not just use words · Embeddings as meaning-vectors · Cosine similarity · **Practical:** tokenize real text & measure semantic similarity

Theory

Tokens: the unit an LLM actually sees

A **token** is a chunk of text — often a whole word, but frequently a sub-word piece. Modern tokenizers (like Byte-Pair Encoding, used by GPT) split text into common fragments. For example, *"tokenization"* might become `token` + `ization`, and an unusual word like *"GroqDoc"* could split into several pieces.

Why sub-words instead of whole words? A vocabulary of every possible word would be gigantic and still miss new words, typos, and other languages. Sub-word tokens give a fixed, manageable vocabulary (say 50,000–100,000 tokens) that can still spell out *anything* by combining pieces.

WHY YOU SHOULD CARE ABOUT TOKENS

Everything is priced and limited in tokens, not words. A model's *context window* (e.g. 128k) is a token budget. API bills are per token. A rough rule of thumb for English: **1 token ≈ 4 characters ≈ ¾ of a word**, so 1,000 tokens is about 750 words.

From tokens to embeddings

Once text is a list of token IDs (just integers), each ID is looked up in an **embedding table** — a big matrix where every token has a learned vector of, say, 768 or 1,536 numbers. That vector is the token's **embedding**.

The magic: these vectors are arranged so that **meaning becomes geometry**. Words used in similar contexts end up close together in this high-dimensional space. "King" sits near "queen" and "throne"; "Python" the language sits near "code" and "script," far from "snake."

Analogy

Imagine a giant library where books aren't shelved alphabetically but by *topic proximity* — cookbooks near each other, physics near maths. An embedding space is that, but with hundreds of dimensions instead of aisles. "Distance" means "difference in meaning."

The classic demonstration is vector arithmetic: `vector("king") − vector("man") + vector("woman")` lands very close to `vector("queen")`. The model captured a gender relationship as a consistent *direction* in space. That's not hard-coded — it emerges from training on how words co-occur.

Two kinds of embeddings you'll meet

Kind	What it represents	Used for
Token embeddings	A single token, static per model	Internal input to a transformer
Sentence / document embeddings	An entire piece of text as one vector	Search, RAG, clustering, similarity (Modules 4 & 5)

Sentence embeddings are what power semantic search: encode a query and a set of documents into vectors, then find the documents whose vectors are closest to the query. This one idea underlies Modules 4 and 5, so it's worth getting comfortable with now.

Measuring closeness: cosine similarity

"Close in meaning" needs a number. The standard is **cosine similarity** — the cosine of the angle between two vectors. It ignores length and focuses on direction:

- **1.0** → same direction (very similar meaning)
- **0.0** → unrelated (perpendicular)
- **−1.0** → opposite

Practical

Two hands-on pieces: first *see* tokenization, then *measure* semantic similarity with real embeddings.

PRACTICAL 2.1 — SEE HOW TEXT IS TOKENIZED

```python
# pip install tiktoken   (OpenAI's fast BPE tokenizer)
import tiktoken

enc = tiktoken.get_encoding("cl100k_base")  # used by GPT-4-class models

texts = [
    "Hello world",
    "tokenization",
    "Sahil is building Aether",
    "antidisestablishmentarianism",
]

for t in texts:
    ids = enc.encode(t)
    pieces = [enc.decode([i]) for i in ids]
    print(f"{t!r}")
    print(f"   token ids : {ids}")
    print(f"   pieces    : {pieces}")
    print(f"   count     : {len(ids)} tokens\n")
```

Notice how short common words are one token, while rare or made-up words shatter into several pieces. That's BPE at work. This is also how you'd estimate cost before an API call — count tokens, don't guess.

PRACTICAL 2.2 — MEANING AS GEOMETRY (SEMANTIC SIMILARITY)

```python
# pip install sentence-transformers
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")  # small, fast, 384-dim

sentences = [
    "How do I train a machine learning model?",
    "What's the best way to fit an ML algorithm?",   # similar meaning
    "I want to bake chocolate chip cookies.",          # unrelated
]

emb = model.encode(sentences)          # shape: (3, 384)

def cosine(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print("sim(0,1) similar meaning  :", round(cosine(emb[0], emb[1]), 3))
print("sim(0,2) unrelated topic  :", round(cosine(emb[0], emb[2]), 3))
```

You'll see the first pair score high (often 0.6–0.8) even though they share almost no words, while the cookie sentence scores low. The model matched *meaning*, not vocabulary — the core trick behind semantic search and RAG.

Try this

Add your own sentences — a question about Python and a question about JavaScript, then a question about cooking. Rank them by similarity to a query. Then try the analogy: encode "king", "man", "woman", "queen" and check that `king - man + woman` is closest to `queen` by cosine similarity.

Pitfall

Embeddings from different models live in different spaces — you **cannot** compare a vector from model A with a vector from model B. Always embed your query and your documents with the *same* model. Mixing them silently produces garbage similarity scores.

Module 2 recap

- **Tokens** are sub-word chunks; they're the real unit of context limits and pricing (≈¾ word each).
- **Embeddings** map tokens/text to vectors where **distance = difference in meaning**.
- **Cosine similarity** turns "how related?" into a number from −1 to 1.
- Sentence embeddings + cosine similarity is the engine under semantic search, vector DBs, and RAG — everything ahead builds on this.

MODULE 03

Prompt Engineering Techniques

> *The model is fixed; your prompt is the one lever you fully control. Prompt engineering is the craft of shaping inputs so you reliably get the output you want.*

IN THIS MODULE

Anatomy of a good prompt · Zero / few-shot · Chain-of-thought · Roles & system prompts · Structured output · **Practical:** the same task, four prompting styles

Theory

What a prompt really is

An LLM predicts the next token from the text so far. A **prompt** is that starting text. So prompt engineering is really about *setting up a context where the most probable continuation is the answer you want.* Everything below is a technique for arranging that context.

The anatomy of a strong prompt

Vague prompts get vague answers. Strong prompts tend to include some of:

Ingredient	Example
Role / persona	"You are a senior Python reviewer."
Clear task	"Review this function for bugs and style."
Context	"It runs in a FastAPI endpoint handling 1000 req/s."
Constraints	"Keep it under 100 words. No external libraries."
Output format	"Return JSON with keys `issue` and `fix`."
Examples	One or two samples of the input→output you expect.

Core techniques

1. Zero-shot

Just ask, no examples. Works well for common tasks the model has seen endlessly. *"Classify this review as positive or negative: 'The food was cold.'"*

2. Few-shot

Show a handful of examples first, then the real input. The model infers the pattern — format, tone, labelling scheme — from your demonstrations. This is the single most reliable way to lock in a specific output style without any training.

3. Chain-of-thought (CoT)

For reasoning, arithmetic, or multi-step logic, ask the model to **“think step by step”** before answering. Forcing intermediate steps dramatically improves accuracy on problems where jumping straight to the answer fails. The reasoning becomes visible — and correct more often.

> **WHY CHAIN-OF-THOUGHT WORKS**
>
> The model has a fixed amount of “compute” per token. Demanding an instant answer to a hard problem crams all the reasoning into one step. Letting it write out steps gives it more tokens — more room to work — before committing to a final answer.

4. Roles & system prompts

Chat models separate a **system** message (persistent instructions and persona) from **user** messages (the actual request). Put durable rules — tone, constraints, output format, who the model is — in the system prompt. It anchors behaviour across the whole conversation.

5. Structured output

When your code consumes the answer, ask for JSON (or a fixed schema) and be explicit: *“Respond with only valid JSON, no markdown, no preamble.”* This makes the output parseable instead of prose you have to scrape.

6. Decomposition

Break a big task into smaller prompts and chain them: extract → plan → draft → refine. Each step is easier to get right, and you can inspect intermediate results. (This is exactly the multi-agent pipeline idea behind a system like Aether.)

Practical

Below is the same sentiment task written four ways, so you can feel the difference. These use the OpenAI-style client, but the *prompt structure* transfers to any provider.

PRACTICAL 3.1 — ZERO-SHOT VS FEW-SHOT VS CHAIN-OF-THOUGHT

```python
# pip install openai   (set OPENAI_API_KEY in your environment)
from openai import OpenAI
client = OpenAI()

def ask(messages):
    r = client.chat.completions.create(
        model="gpt-4o-mini", messages=messages, temperature=0,
    )
    return r.choices[0].message.content

review = "The battery dies in an hour but the screen is gorgeous."

# --- Zero-shot ---
zero = ask([{"role": "user",
    "content": f"Classify sentiment as positive/negative/mixed: {review}"}])

# --- Few-shot: teach the exact format with examples ---
few = ask([
    {"role": "user", "content":
     "Review: 'Fast and cheap.' -> positive\n"
     "Review: 'Broke on day one.' -> negative\n"
     f"Review: '{review}' ->"}
])

# --- Chain-of-thought: force reasoning before the label ---
cot = ask([{"role": "user",
    "content": f"Review: {review}\n"
        "Think step by step about the pros and cons, "
        "then end with 'Label: '."}])

print("ZERO-SHOT:\n", zero)
print("\nFEW-SHOT:\n", few)
print("\nCHAIN-OF-THOUGHT:\n", cot)
```

PRACTICAL 3.2 — SYSTEM PROMPT + GUARANTEED JSON OUTPUT

```python
import json

messages = [
    {"role": "system", "content":
        "You are a strict product-review analyst. "
        "Always reply with ONLY valid JSON, no markdown, no extra text."},
    {"role": "user", "content":
        f"Analyze this review and return keys "
        '"sentiment", "positives" (list), "negatives" (list): '
        f"{review}"},
]

raw = ask(messages)
data = json.loads(raw)          # now it's a real Python dict
print(data["sentiment"])
print("Cons:", data["negatives"])
```

> Try this

Run 3.1 on a genuinely tricky, sarcastic review (“Oh great, another update that breaks everything”). Zero-shot often trips on sarcasm; chain-of-thought usually recovers. That gap *is* the value of the technique. Then in 3.2, remove the “ONLY valid JSON” line and watch `json.loads` break — that’s why the instruction matters.

Pitfalls

Over-prompting: piling on ten conflicting instructions confuses the model — be clear, not verbose. **Assuming determinism:** set `temperature=0` for consistent, testable output; higher values add creativity *and* randomness. **Trusting format by hope:** if you need JSON, say so explicitly and still wrap `json.loads` in a try/except.

Module 3 recap

- A prompt sets up a context so the *most likely continuation* is your desired answer.
- **Few-shot** locks format via examples; **chain-of-thought** boosts reasoning by giving the model room to think.
- Use **system prompts** for durable rules and **explicit format instructions** for machine-readable output.
- Set `temperature=0` when you need reproducibility; decompose hard tasks into chained steps.

MODULE 04

Retrieval-Augmented Generation

> *An LLM only knows what it was trained on — not your company docs, not last week's news, not your codebase. RAG fixes that by fetching relevant information at question-time and feeding it to the model.*

IN THIS MODULE

Why RAG exists · The pipeline: chunk → embed → store → retrieve → generate · **Practical:** a complete minimal RAG in ~50 lines

Theory

The problem RAG solves

Three limits of a bare LLM:

- **Knowledge cutoff** — it doesn't know anything after training.
- **No private data** — it never saw your PDFs, tickets, or wiki.
- **Hallucination** — asked something it doesn't know, it may confidently invent an answer.

You *could* retrain the model on your data, but that's expensive, slow, and instantly stale. **RAG** takes a cheaper route: keep the model as-is, and at question-time *retrieve* the most relevant snippets from your own data and paste them into the prompt as context. The model then answers *grounded in* that context.

Analogy

A closed-book exam vs an open-book exam. A bare LLM answers from memory (closed book). RAG hands it the exact relevant pages right before it answers (open book). Same brain — far fewer made-up answers.

The pipeline

RAG has two phases. You do **indexing** once (or whenever data changes), and **retrieval + generation** on every query.

Phase A — Indexing (offline, done once)

Step	What happens
1. Load	Read your sources — PDFs, docs, web pages, database rows.
2. Chunk	Split long text into small passages (e.g. 200–500 words), often with slight overlap so ideas aren't cut mid-thought.
3. Embed	Turn each chunk into a vector (Module 2) using an embedding model.
4. Store	Save vectors + text in a vector database (Module 5) for fast similarity search.

Phase B — Retrieval + Generation (every query)

Step	What happens
5. Embed query	Turn the user's question into a vector with the *same* model.
6. Retrieve	Find the top-*k* chunks whose vectors are most similar to the query.
7. Augment	Insert those chunks into the prompt as context.
8. Generate	Ask the LLM to answer *using only* that context, and cite it.

> **THE HEART OF RAG IN ONE LINE**
>
> **Retrieved context + user question → prompt → LLM.** Everything else is engineering around getting the *right* chunks into that prompt.

Why chunking matters more than it looks

Chunk too big and each retrieval drags in irrelevant text, diluting the answer and wasting tokens. Chunk too small and you sever the context a sentence needs to make sense. Overlap (repeating a sentence or two between neighbours) prevents important ideas from being split across a boundary. Good chunking is often the difference between a RAG system that works and one that doesn't.

Practical

A complete, dependency-light RAG. We'll skip a real vector DB here (that's Module 5) and do the similarity search by hand with NumPy so you can see every step. Same logic, just transparent.

PRACTICAL 4.1 — A MINIMAL END-TO-END RAG

```python
# pip install sentence-transformers openai numpy
from sentence_transformers import SentenceTransformer
from openai import OpenAI
import numpy as np

embedder = SentenceTransformer("all-MiniLM-L6-v2")
client = OpenAI()

# --- Phase A: indexing (done once) ---
docs = [
    "Aether turns a natural-language prompt into a static Next.js app.",
    "Aether's pipeline uses agents: requirement extraction, planning, design, codegen.",
    "The Aether backend is built with FastAPI and shared Pydantic schemas.",
    "The orchestrator runs agents sequentially and can resume from failure.",
    "Photosynthesis converts sunlight into chemical energy in plants.",  # distractor
]
doc_vecs = embedder.encode(docs)                  # (5, 384)

def cosine(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def retrieve(query, k=3):
    q = embedder.encode([query])[0]
    scores = [cosine(q, d) for d in doc_vecs]
    top = np.argsort(scores)[::-1][:k]          # highest first
    return [docs[i] for i in top]

# --- Phase B: retrieval + generation (every query) ---
def rag_answer(question):
    context = "\n".join(f"- {c}" for c in retrieve(question))
    prompt = (
        "Answer using ONLY the context below. "
        "If the answer isn't there, say 'I don't know.'\n\n"
        f"Context:\n{context}\n\nQuestion: {question}"
    )
    r = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0,
    )
    return r.choices[0].message.content

print(rag_answer("What backend does Aether use?"))
print(rag_answer("Can the orchestrator recover from a crash?"))
print(rag_answer("What is the capital of France?"))   # should say I don't know
```

The third question is the important test. A bare LLM knows Paris — but our grounded prompt has no France context, so a well-behaved RAG system answers *"I don't know."* That refusal is a **feature**: it means the model is sticking to your data instead of guessing.

> Try this
> 1. Print the retrieved chunks inside `rag_answer` to see which docs won.
> 2. Ask "What powers photosynthesis?" — the distractor gets retrieved and the model answers from it. That shows retrieval quality directly controls answer quality.
> 3. Lower `k` to 1 and see answers get thinner; raise it to 5 and watch irrelevant context creep in.

Pitfalls

“RAG eliminates hallucination.” It reduces it — the model can still misread context or answer beyond it. Always instruct it to stick to the context and, ideally, cite sources. **Garbage in, garbage out:** if retrieval fetches the wrong chunks, generation can’t save you. Most RAG debugging is retrieval debugging, not prompt tweaking.

Module 4 recap

- RAG grounds an LLM in **your** data without retraining — open-book instead of closed-book.
- Index once: **load** → **chunk** → **embed** → **store**. Per query: **embed** → **retrieve top-k** → **augment prompt** → **generate**.
- Retrieval quality is everything; chunking strategy quietly decides success.
- Tell the model to answer only from context so it says “I don’t know” instead of hallucinating.

MODULE 05

Vector Databases Overview

> *In Module 4 we compared the query against every document with a loop. That's fine for five docs and hopeless for five million. Vector databases make similarity search fast at scale.*

IN THIS MODULE

What they store · The scale problem · ANN & indexes (HNSW) · Metadata filtering · The landscape · **Practical:** real search with ChromaDB

Theory

What a vector database is

A normal database finds rows by exact match: *"give me the user where id = 42."* A **vector database** finds rows by *similarity*: *"give me the 5 items whose embedding is closest to this vector."* It stores embeddings (Module 2) and is optimised for one operation: **nearest-neighbour search** in high-dimensional space.

Why you can't just loop

Comparing a query to every stored vector — *brute-force / exact search* — is O(n) per query. With millions of vectors of 768+ dimensions, that's far too slow for real-time use. The solution is **Approximate Nearest Neighbour (ANN)** search: give up a tiny bit of accuracy to go dramatically faster.

> **THE CORE TRADE-OFF**
>
> **Exact search** = perfect results, slow. **ANN** = "probably the right neighbours" in milliseconds, even over billions of vectors. For search and RAG, 99% accuracy at 100× the speed is almost always the right call.

How ANN indexes work (the intuition)

Instead of checking everything, ANN indexes build a clever structure so search only visits a small, promising slice of the data. The most common is **HNSW** (Hierarchical Navigable Small World):

> Analogy for HNSW
>
> Think of finding a house. You don't check every address. You take the highway to the right city (top layer), exit to the right neighbourhood (middle layer), then walk the right street (bottom layer). HNSW builds those layers of "shortcuts" so search zooms in fast, hopping from far-away to nearby vectors in a few jumps.

Other index families you'll hear about: **IVF** (cluster vectors, only search the nearest clusters) and **PQ** (compress vectors to save memory). You rarely implement these — you pick them as settings.

Metadata filtering: the underrated feature

Real queries aren't purely semantic. You often want *"the most relevant chunks **from this user's documents**, written **after January**."* Vector DBs let you attach metadata (tags, dates, user IDs) to each vector and filter on it *alongside* similarity search. This is essential for multi-tenant apps and permissions.

The landscape

Tool	Best for
ChromaDB	Local dev, prototyping, learning. Runs in-process, zero setup.
FAISS	A library (not a server) from Meta; blazing fast, you manage storage yourself.
Pinecone	Fully managed cloud service; scale without ops work.
Weaviate / Qdrant / Milvus	Open-source, production-grade, self-hostable, rich filtering.
pgvector	A Postgres extension — add vectors to a database you already run.

> **DO YOU EVEN NEED ONE?**
>
> For a few thousand vectors, a NumPy array and cosine similarity (Module 4) is genuinely fine — simpler and fast enough. Reach for a vector DB when you have *lots* of vectors, need persistence, want metadata filtering, or need concurrent access. Don't add infrastructure you don't need yet.

Practical

ChromaDB gives you a real vector store in a few lines, and it can even run the embedding model for you. We'll rebuild Module 4's search — this time with a proper index and metadata filtering.

PRACTICAL 5.1 — STORE, SEARCH, AND FILTER WITH CHROMADB

```python
# pip install chromadb
import chromadb

client = chromadb.Client()                    # in-memory; use PersistentClient to save
collection = client.create_collection("aether_docs")

# Add documents. Chroma embeds them automatically with a default model.
collection.add(
    documents=[
        "Aether turns prompts into static Next.js apps.",
        "Aether uses a multi-agent pipeline for code generation.",
        "The backend runs on FastAPI with Pydantic schemas.",
        "The orchestrator can resume from failure.",
    ],
    metadatas=[
        {"topic": "overview"},
        {"topic": "pipeline"},
        {"topic": "backend"},
        {"topic": "pipeline"},
    ],
    ids=["d1", "d2", "d3", "d4"],
)

# 1) Pure semantic search: top-2 closest to the query.
res = collection.query(query_texts=["How does the codegen work?"], n_results=2)
print("Semantic hits:", res["documents"][0])

# 2) Semantic search + metadata filter: only 'pipeline' docs.
res2 = collection.query(
    query_texts=["agents"],
    n_results=2,
    where={"topic": "pipeline"},          # the killer feature
)
print("Filtered hits:", res2["documents"][0])
```

Notice how little code that took. Chroma handled embedding, indexing, storage, and the similarity search. Swapping this into the RAG system from Module 4 — just replace the NumPy `retrieve()` with `collection.query()` — turns a toy into something that scales.

Try this

1. Switch `chromadb.Client()` to `chromadb.PersistentClient(path="./db")`, run once, then restart and query without re-adding — the data survives.
2. Add 20+ documents with a `"date"` metadata field and filter with `where={"date": {"$gte": "2026-01-01"}}`.
3. Bolt this onto the Module 4 `rag_answer` function to build a persistent, filterable RAG.

Pitfall

If you embed your documents with model A but let the DB embed queries with model B, similarity is meaningless (Module 2's warning, at scale). When you bring your own embeddings, make sure the query uses the exact same model — and the same vector dimension, or the DB will reject it.

Module 5 recap

- Vector DBs specialise in **nearest-neighbour search** over embeddings — find by meaning, not exact match.
- **ANN** indexes (like **HNSW**) trade a sliver of accuracy for enormous speed at scale.
- **Metadata filtering** combines “most similar” with “matching these constraints” — vital for real apps.
- Small dataset? NumPy is fine. Scale, persistence, or filtering? Reach for Chroma, Qdrant, pgvector, Pinecone, etc.

MODULE 06

LangChain Basics

> *You've now built prompts, RAG, and vector search by hand. LangChain is a framework that gives these pieces a common interface so you can wire them together — and swap parts — without rewriting everything.*

IN THIS MODULE

What LangChain is (and isn't) · Core building blocks · LCEL and the pipe operator · **Practical:** a prompt→model→parser chain, then a mini RAG chain

Theory

What LangChain is

LangChain is a toolkit for building LLM applications. Its value isn't any single feature — it's **standard interfaces**. Every chat model exposes the same methods; every vector store exposes the same search call; every prompt template works the same way. So you can build a pipeline and later swap OpenAI for a local model, or Chroma for Pinecone, by changing one line.

WHAT IT IS / ISN'T

LangChain **is** glue and abstractions for composing LLM components. It **is not** a model — it calls models you provide. Think of it as the plumbing that connects prompts, models, retrievers, and tools into one flow.

Core building blocks

Block	Role
Chat models	A uniform wrapper over any LLM (`ChatOpenAI` , `ChatAnthropic` , local models).
Prompt templates	Reusable prompts with `{placeholders}` filled at runtime.
Output parsers	Turn the raw model reply into a string, JSON, or a typed object.
Retrievers	A standard "fetch relevant docs" interface over any vector store.
Chains	Compositions that pipe these blocks together into one callable.
Agents	Chains that decide *which* tools to call (Module 7).

LCEL: the pipe that ties it together

Modern LangChain uses **LCEL** (LangChain Expression Language). You compose components with the `|` operator, exactly like a Unix pipe: the output of one becomes the input of the next.

`prompt | model | parser` reads left to right: fill the prompt, send it to the model, parse the reply. This gives you streaming, batching, and async support for free, and makes the data flow obvious at a glance.

> Analogy
> LCEL is a factory conveyor belt. Raw input enters, passes through stations (prompt formatter, then model, then parser), and a finished product comes out the end. Adding or replacing a station is just editing one link in the chain.

Practical

Two chains. First the canonical `prompt | model | parser`; then a small RAG chain that reuses Module 4's idea with LangChain's plumbing.

PRACTICAL 6.1 — YOUR FIRST LCEL CHAIN

```
# pip install langchain langchain-openai
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

model  = ChatOpenAI(model="gpt-4o-mini", temperature=0)
prompt = ChatPromptTemplate.from_template(
    "Explain {topic} to a beginner in exactly 2 sentences."
)
parser = StrOutputParser()

# Compose with the pipe operator -- this IS the chain.
chain = prompt | model | parser

print(chain.invoke({"topic": "vector databases"}))

# Same chain, run on many inputs at once:
for out in chain.batch([{"topic": "RAG"}, {"topic": "tokens"}]):
    print("-", out)
```

That's the whole idea: three components, one `|`-chain, and you get `invoke`, `batch`, and streaming without extra code. Swap `ChatOpenAI` for `ChatAnthropic` and nothing else changes.

PRACTICAL 6.2 — A RAG CHAIN, THE LANGCHAIN WAY

```python
from langchain_core.runnables import RunnablePassthrough

# A trivial 'retriever' function. In practice this is a real vector store.
knowledge = [
    "Aether's backend uses FastAPI and Pydantic.",
    "Aether generates static Next.js frontends from prompts.",
    "The orchestrator resumes from failure and can pause for clarification.",
]

def retrieve(question):
    # naive keyword match; swap for vectorstore.as_retriever() in real code
    hits = [d for d in knowledge if any(
        w in d.lower() for w in question.lower().split())]
    return "\n".join(hits) or "(no context found)"

rag_prompt = ChatPromptTemplate.from_template(
    "Use ONLY this context to answer.\n\n"
    "Context:\n{context}\n\nQuestion: {question}"
)

# Build inputs for the prompt: context from retrieve(), question passed through.
rag_chain = (
    {"context": (lambda x: retrieve(x["question"])),
     "question": RunnablePassthrough() | (lambda x: x["question"])}
    | rag_prompt
    | model
    | parser
)

print(rag_chain.invoke({"question": "What backend does Aether use?"}))
```

The dictionary at the front builds the two prompt variables in parallel — `context` comes from retrieval, `question` passes straight through — then the same `prompt | model | parser` tail finishes the job. This is exactly the RAG pattern from Module 4, expressed as a composable chain.

Try this

Replace the naive `retrieve` with a real Chroma retriever from Module 5: `vectorstore.as_retriever(search_kwargs={"k": 3})`. LangChain retrievers plug straight into a chain, so you get production-grade retrieval with the same three-line tail.

Pitfall

LangChain's API has changed a lot across versions — older tutorials use `LLMChain` and `initialize_agent`, which are legacy. Prefer **LCEL** (the `|` style) and the split packages (`langchain-openai`, `langchain-core`). When you hit an import error, check the version the tutorial was written for. Also: LangChain is powerful but adds abstraction — for a simple single call, the plain SDK from Module 3 is often clearer.

Module 6 recap

- LangChain provides **standard interfaces** so components (models, prompts, retrievers) are swappable.
- **LCEL** composes them with `|`: `prompt | model | parser`, giving invoke/batch/stream for free.
- RAG becomes a chain: build `{context, question}`, pipe through prompt → model → parser.
- Use modern LCEL and split packages; skip the framework entirely when a single plain SDK call would do.

MODULE 07

AI Agents Overview

> *So far the LLM just talks. An agent lets it **act** — decide what to do, use tools, observe results, and loop until a goal is met. This is where LLMs stop being chatbots and start being workers.*

IN THIS MODULE

Chain vs agent · Tools · The reasoning loop · ReAct · Multi-agent systems · **Practical:** build a working tool-using agent loop

Theory

Chain vs agent

A **chain** (Module 6) is a fixed pipeline: the steps are decided by you, in advance. An **agent** decides the steps *itself*, at runtime, based on what it's seeing. Give it a goal and a set of tools, and it reasons about which tool to use, uses it, looks at the result, and repeats until done.

> Analogy
> A chain is a recipe — fixed steps, same every time. An agent is a chef with a goal ("make dinner") and a kitchen of tools — it decides what to chop, taste, and adjust as it goes, reacting to what it finds.

Tools: the agent's hands

An LLM can't browse the web, run code, or query a database on its own — it only produces text. **Tools** bridge that gap. A tool is just a function (search the web, run a calculation, call an API, look up a document) that you describe to the model. The model can then *request* a tool call by outputting its name and arguments; your code runs it and feeds the result back.

The reasoning loop

Nearly every agent runs the same loop:

Step	What happens
1. Think	The LLM reasons about the goal and what it needs next.
2. Act	It chooses a tool and the arguments to call it with.
3. Observe	Your code runs the tool and returns the result to the model.
4. Repeat	The model decides: goal met → answer; else → go back to step 1.

REACT = REASON + ACT

The most common agent pattern is **ReAct**: the model interleaves *reasoning* (“I need today’s exchange rate”) with *actions* (call the currency tool), using each observation to inform the next thought. Reason, act, observe, repeat — until it can answer.

Multi-agent systems

Complex jobs are often split across several specialised agents, each with a narrow role, coordinated by an orchestrator. That’s precisely the shape of a system like **Aether**: a requirement-extraction agent, a planning agent, a design agent, and a codegen agent, run in sequence by an orchestrator. Each agent is simpler and more reliable than one giant do-everything prompt, and you can inspect and retry each stage independently.

Practical

We’ll build the loop by hand so nothing is hidden, using the model’s native tool-calling. Two tools: a calculator and a tiny knowledge lookup. Watch the model choose between them.

PRACTICAL 7.1 — A TOOL-USING AGENT LOOP FROM SCRATCH

```python
# pip install openai
from openai import OpenAI
import json
client = OpenAI()

# --- 1) Define the actual tool functions ---
def calculator(expression):
    return str(eval(expression))          # demo only; never eval untrusted input

def lookup_aether(topic):
    facts = {"backend": "FastAPI + Pydantic",
             "frontend": "static Next.js"}
    return facts.get(topic.lower(), "unknown")

available = {"calculator": calculator, "lookup_aether": lookup_aether}

# --- 2) Describe the tools to the model ---
tools = [
    {"type": "function", "function": {
        "name": "calculator",
        "description": "Evaluate a math expression.",
        "parameters": {"type": "object",
            "properties": {"expression": {"type": "string"}},
            "required": ["expression"]}}},
    {"type": "function", "function": {
        "name": "lookup_aether",
        "description": "Look up a fact about the Aether project.",
        "parameters": {"type": "object",
            "properties": {"topic": {"type": "string"}},
            "required": ["topic"]}}},
]

# --- 3) The agent loop ---
def run_agent(question, max_steps=5):
    messages = [{"role": "user", "content": question}]
    for _ in range(max_steps):
        resp = client.chat.completions.create(
            model="gpt-4o-mini", messages=messages, tools=tools,
        ).choices[0].message
        messages.append(resp)

        if not resp.tool_calls:           # no tool wanted -> we're done
            return resp.content

        for call in resp.tool_calls:      # THINK->ACT already happened; now OBSERVE
            fn   = available[call.function.name]
            args = json.loads(call.function.arguments)
            result = fn(**args)
            print(f"  [tool] {call.function.name}({args}) -> {result}")
            messages.append({"role": "tool",
                "tool_call_id": call.id, "content": result})
    return "(stopped: too many steps)"

print(run_agent("What is 145 * 12, and what frontend does Aether use?"))
```

That single question needs *two different tools*. The agent figures that out on its own: it calls `calculator` for the math and `lookup_aether` for the fact, observes both results, then writes a combined answer. You never told it the order — that's the difference between an agent and a chain.

Try this

1. Add a `get_weather(city)` tool (return a hard-coded string) and ask a 3-part question — watch it chain three tool calls.
2. Print `messages` at the end to see the full Think→Act→Observe trace.
3. Ask something needing no tools ("Say hello") and confirm it answers immediately without calling one.

Pitfalls

Runaway loops: always cap steps (the `max_steps` guard) — a confused agent can loop forever and burn tokens. **Unsafe tools:** `eval` here is for demonstration only; never run model-generated code or SQL without sandboxing and validation. **Over-agentifying:** if the task is a fixed sequence, a chain is simpler, cheaper, and more predictable than an agent. Reach for agents only when the path genuinely varies.

Module 7 recap

- An **agent** chooses its own steps at runtime; a **chain** follows a fixed path you defined.
- **Tools** give the LLM hands — functions it can request to run.
- The **ReAct loop** is Think → Act → Observe → repeat, until the goal is met.
- Split hard jobs into **multiple specialised agents** (the Aether pattern); always cap steps and sandbox risky tools.

MODULE 08

Responsible AI

> *Building something that works is half the job. Building something that's fair, safe, private, and honest is the other half — and it's what separates a demo from a product people can trust.*

IN THIS MODULE

Bias · Hallucination · Privacy · Safety & misuse · Transparency · Practical guardrails · **Practical:** input filtering, grounding checks, PII scrubbing

Theory

Why this matters technically, not just ethically

LLMs learn from vast human text, so they absorb its patterns — including its mistakes and biases. They generate fluent text whether or not it's true. They'll repeat back whatever you feed them, including secrets. None of this is malice; it's how the technology works. Responsible AI is the engineering discipline of anticipating these failure modes and designing around them.

The main risks

Risk	What it looks like	Why it happens
Bias	Skewed or stereotyped outputs across gender, race, etc.	Training data reflects real-world imbalances.
Hallucination	Confident, fluent, false statements.	The model optimises for plausible text, not truth.
Privacy	Leaking personal or confidential data.	Data can appear in training sets or user prompts.
Safety / misuse	Helping with harmful or illegal requests.	A helpful model can be steered toward harm.
Opacity	"Why did it say that?" is hard to answer.	Billions of parameters; no simple audit trail.

Principles to design by

- **Fairness** — test outputs across groups; don't ship a model you've only checked on people like yourself.
- **Transparency** — tell users they're talking to AI; cite sources (RAG helps here); be honest about limits.
- **Privacy** — minimise what you collect, scrub sensitive data, and know what your provider does with prompts.

- **Accountability** — a human owns the outcome. Keep humans in the loop for high-stakes decisions (medical, legal, financial, hiring).
- **Safety** — filter inputs and outputs; refuse harmful requests; monitor for abuse.

> **THE GROUNDING PRINCIPLE**
>
> The single most effective anti-hallucination tactic you already know: **RAG**. Grounding answers in retrieved sources — and instructing the model to say "I don't know" when the sources don't cover it — converts confident guessing into honest, checkable answers.

Human-in-the-loop

For any consequential decision, the model should *assist*, not *decide*. Draft the email, but a person sends it. Flag the anomaly, but a person acts. Suggest the diagnosis, but a clinician confirms. This isn't a limitation to engineer away — it's the design.

Practical

Responsible AI isn't only policy — a lot of it is code you add around the model. Here are three practical guardrails you can drop into any app.

PRACTICAL 8.1 — INPUT SAFETY FILTER (A SIMPLE GUARDRAIL)

```
from openai import OpenAI
client = OpenAI()

def is_safe(user_text):
    # OpenAI's moderation endpoint flags hate, violence, self-harm, etc.
    r = client.moderations.create(
        model="omni-moderation-latest", input=user_text)
    return not r.results[0].flagged

def safe_chat(user_text):
    if not is_safe(user_text):
        return "Sorry, I can't help with that request."
    r = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": user_text}])
    return r.choices[0].message.content

print(safe_chat("Explain gradient descent simply."))   # allowed
```

PRACTICAL 8.2 — SCRUB PII BEFORE IT EVER REACHES THE MODEL

```python
import re

def redact_pii(text):
    text = re.sub(r"[\w.+-]+@[\w-]+\.[\w.-]+", "[EMAIL]", text)
    text = re.sub(r"\b\d{10}\b", "[PHONE]", text)
    text = re.sub(r"\b\d{4}[\s-]?\d{4}[\s-]?\d{4}\b", "[CARD]", text)
    return text

raw = "Email me at sahil@example.com or call 9876543210."
print(redact_pii(raw))
# -> "Email me at [EMAIL] or call [PHONE]."
```

PRACTICAL 8.3 — A GROUNDING / HALLUCINATION CHECK

```python
def grounded_answer(question, context):
    # Force the model to admit ignorance instead of inventing.
    prompt = (
        "Answer strictly from the context. If it is not covered, "
        "reply exactly: 'Not in the provided sources.'\n\n"
        f"Context: {context}\n\nQuestion: {question}"
    )
    r = client.chat.completions.create(
        model="gpt-4o-mini", temperature=0,
        messages=[{"role": "user", "content": prompt}])
    return r.choices[0].message.content

ctx = "Aether uses a FastAPI backend."
print(grounded_answer("What backend does Aether use?", ctx))   # answers
print(grounded_answer("Who founded Google?", ctx))              # refuses
```

Try this

Combine all three into one pipeline: `redact_pii` → `is_safe` → `grounded_answer`. That ordering — scrub, screen, then ground — is a realistic minimal safety layer for a production RAG app. Add logging so you can audit what was flagged and why.

Pitfall

Guardrails are necessary but never sufficient. Regex misses novel PII formats; moderation misses cleverly-worded harm; grounding prompts can still be overridden by a determined user. Treat these as layers of defence, keep a human accountable for outcomes, and never market an LLM feature as “100% safe” or “always accurate.”

Module 8 recap

- LLMs inherit **bias**, **hallucinate**, and can **leak data** — by design, not by accident.
- Design for **fairness, transparency, privacy, accountability, safety**; keep humans in the loop for high-stakes calls.
- **RAG + “say I don’t know”** is your strongest practical anti-hallucination tool.
- Add real guardrails in code: **input filtering, PII scrubbing, grounding checks** — layered, logged, and never assumed perfect.

WRAPPING UP

How it all connects

> *Eight modules, one system. Here's the whole stack in a single mental model — and where to go next.*

The big picture

Everything you learned fits together into one flow. A modern GenAI application is essentially this stack:

Layer	Module	Its job
The engine	1 — Transformers	The architecture that makes it all possible.
The language	2 — Tokens & Embeddings	Turning text into numbers and meaning into geometry.
The controls	3 — Prompt Engineering	Steering the model with well-shaped input.
The memory	4 & 5 — RAG + Vector DBs	Grounding answers in your own, up-to-date data.
The wiring	6 — LangChain	Composing components into maintainable pipelines.
The autonomy	7 — Agents	Letting the model plan and act with tools.
The conscience	8 — Responsible AI	Keeping it fair, private, safe, and honest.

YOUR CAPSTONE

You now have every piece to build a real, grounded, safe assistant: chunk and embed a set of documents (2), store them in a vector DB (5), retrieve and generate with RAG (4) using strong prompts (3), wire it with LangChain (6), give it tools so it can act (7), and wrap it in guardrails (8) — all running on transformers (1). That's a portfolio project. Build it.

Suggested next steps

- **Build the capstone above** end-to-end over your own notes or the Aether docs — it cements every module at once.
- **Read the source:** the "Attention Is All You Need" paper, and Jay Alammar's "The Illustrated Transformer" for visual intuition.
- **Go deeper on evaluation:** how do you *measure* whether a RAG or agent is actually good? That's the next frontier once you can build one.
- **Practice explaining each module aloud in two minutes** — if you can teach it simply, you own it. Perfect prep for a viva or interview.

You don’t need to memorise every line of code here. Understand the ideas, keep this as a reference, and let the practicals be muscle memory you rebuild whenever you need them. Come back to any module, run the code, break it, fix it — that loop is where the learning actually happens.